



Lab Assignment 4 — NLP Preprocessing and Text Classification

Objective: Implement NLP preprocessing techniques and build a text classification model using machine learning and deep learning techniques.

Learning Outcomes:

- Apply NLP preprocessing (tokenization, stopwords removal, stemming, lemmatization)
- Implement text vectorization (TF-IDF, CountVectorizer)
- Build and compare multiple ML & DL classification models
- Evaluate model performance using metrics and visualizations

Dataset: 20 Newsgroups (multi-class text classification) — a standard NLP benchmark dataset

Author: [Rohit Thorat] | **Roll No:** [202402070041] | **Date:** 10 April 2026



Section 1: Install & Import Dependencies

```
# Install required libraries (run once in Colab)
!pip install nltk scikit-learn matplotlib seaborn wordcloud tensorflow keras --

# Download NLTK resources
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')

print("✅ All dependencies installed and NLTK data downloaded.")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
```

```
[nltk_data] Unzipping tokenizers/punkt_tab.zip.  
✔ All dependencies installed and NLTK data downloaded.
```

```
# — Core Libraries —————  
import numpy as np  
import pandas as pd  
import re  
import warnings  
warnings.filterwarnings('ignore')  
  
# — NLP Libraries —————  
from nltk.tokenize import word_tokenize, sent_tokenize  
from nltk.corpus import stopwords  
from nltk.stem import PorterStemmer, SnowballStemmer  
from nltk.stem import WordNetLemmatizer  
  
# — Sklearn —————  
from sklearn.datasets import fetch_20newsgroups  
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer  
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV  
from sklearn.naive_bayes import MultinomialNB  
from sklearn.linear_model import LogisticRegression, SGDClassifier  
from sklearn.svm import LinearSVC  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import (  
    accuracy_score, classification_report, confusion_matrix,  
    f1_score, precision_score, recall_score  
)  
from sklearn.utils.class_weight import compute_class_weight  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import LabelEncoder  
  
# — Deep Learning —————  
import tensorflow as tf  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import (  
    Dense, Dropout, Embedding, LSTM, Bidirectional,  
    GlobalMaxPooling1D, Conv1D, MaxPooling1D  
)  
from tensorflow.keras.preprocessing.text import Tokenizer  
from tensorflow.keras.preprocessing.sequence import pad_sequences  
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau  
from tensorflow.keras.utils import to_categorical  
from tensorflow.keras.regularizers import l2  
  
# — Visualization —————  
import matplotlib.pyplot as plt  
import seaborn as sns  
from wordcloud import WordCloud  
from collections import Counter
```

```
# — Plot Style —————
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.family'] = 'DejaVu Sans'
```

```
print(f"✅ TensorFlow version: {tf.__version__}")
print(f"✅ All imports successful.")
```

✅ TensorFlow version: 2.19.0

✅ All imports successful.

Section 2: Dataset Loading & Exploratory Data Analysis (EDA)

```
# — Load 20 Newsgroups Dataset —————
# We use 6 representative categories to keep training manageable in Colab
CATEGORIES = [
    'rec.sport.hockey',
    'sci.med',
    'sci.space',
    'talk.politics.guns',
    'comp.graphics',
    'rec.autos'
]

# Load train and test splits
newsgroups_train = fetch_20newsgroups(
    subset='train', categories=CATEGORIES,
    remove=('headers', 'footers', 'quotes'),
    random_state=42
)
newsgroups_test = fetch_20newsgroups(
    subset='test', categories=CATEGORIES,
    remove=('headers', 'footers', 'quotes'),
    random_state=42
)

# — IMPORTANT: sklearn assigns numeric labels ALPHABETICALLY, not in
# the order of the CATEGORIES list above. Always use target_names[label]
# for correct label-to-category-name mapping.
#
# Alphabetical assignment:
# 0 → comp.graphics
# 1 → rec.autos
# 2 → rec.sport.hockey
# 3 → sci.med
# 4 → sci.space
```

```
# 5 → talk.politics.guns
target_names = newsgroups_train.target_names # ground-truth label→name mappir
NUM_CLASSES = len(target_names)

print("Label → Category mapping (as assigned by sklearn, alphabetical order):")
for i, name in enumerate(target_names):
    print(f"Label {i} → {name}")

# Build DataFrames using target_names for consistency
df_train = pd.DataFrame({
    'text' : newsgroups_train.data,
    'label' : newsgroups_train.target,
    'category': [target_names[t] for t in newsgroups_train.target]
})
df_test = pd.DataFrame({
    'text' : newsgroups_test.data,
    'label' : newsgroups_test.target,
    'category': [target_names[t] for t in newsgroups_test.target]
})

# short_labels derived from target_names (alphabetical order = label order)
short_labels = [c.split('.')[0] for c in target_names]

y_train = df_train['label'].values
y_test = df_test['label'].values

print(f"\nTraining samples : {len(df_train)}")
print(f"Test samples : {len(df_test)}")
print(f"short_labels : {short_labels}")
print("\nSample entry:")
print(df_train['text'].iloc[0][:300])
```

Label → Category mapping (as assigned by sklearn, alphabetical order):

Label 0 → comp.graphics
 Label 1 → rec.autos
 Label 2 → rec.sport.hockey
 Label 3 → sci.med
 Label 4 → sci.space
 Label 5 → talk.politics.guns

Training samples : 3511

Test samples : 2338

short_labels : ['graphics', 'autos', 'hockey', 'med', 'space', 'guns']

Sample entry:

Joint Press release ESA/UN No 18-93
 Paris, 19 April 1993

UN/ESA joint training course on satellite applications
 to be held in Italy, 19-30 April

The United Nations and the European Space Agency (ESA)

are jointly organising a training course on the applications of satellite data gathered by the Eur

```
# — EDA Visualizations —————
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Exploratory Data Analysis – 20 Newsgroups', fontsize=15, fontweig

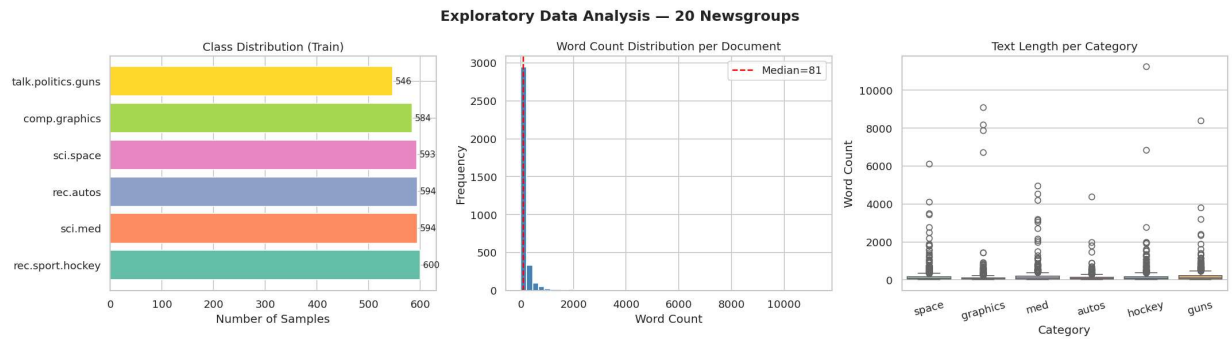
# 1. Class distribution
cat_counts = df_train['category'].value_counts()
axes[0].barh(cat_counts.index, cat_counts.values,
              color=sns.color_palette('Set2', NUM_CLASSES))
axes[0].set_title('Class Distribution (Train)')
axes[0].set_xlabel('Number of Samples')
for i, v in enumerate(cat_counts.values):
    axes[0].text(v + 5, i, str(v), va='center', fontsize=9)

# 2. Text length distribution
df_train['text_len'] = df_train['text'].apply(lambda x: len(x.split()))
axes[1].hist(df_train['text_len'], bins=50, color='steelblue', edgecolor='white')
axes[1].set_title('Word Count Distribution per Document')
axes[1].set_xlabel('Word Count')
axes[1].set_ylabel('Frequency')
med_val = df_train['text_len'].median()
axes[1].axvline(med_val, color='red', linestyle='--', label=f'Median={med_val:.')
axes[1].legend()

# 3. Box plot of text length per class
df_train['short_cat'] = df_train['category'].apply(lambda x: x.split('.')[0])
sns.boxplot(data=df_train, x='short_cat', y='text_len', palette='Set3', ax=axes)
axes[2].set_title('Text Length per Category')
axes[2].set_xlabel('Category')
axes[2].set_ylabel('Word Count')
axes[2].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.show()

print("\nDataFrame Info:")
print(df_train[['text', 'category', 'text_len']].describe(include='all'))
```



DataFrame Info:

	text	category	text_len
count	3511	3511	3511.000000
unique	3412	6	NaN
top		rec.sport.hockey	NaN
freq	80	600	NaN
mean	NaN	NaN	181.618058
std	NaN	NaN	504.588898
min	NaN	NaN	0.000000
25%	NaN	NaN	41.000000
50%	NaN	NaN	81.000000
75%	NaN	NaN	158.000000
max	NaN	NaN	11251.000000

✎ Section 3: NLP Preprocessing Pipeline

We implement and **compare** each preprocessing step individually before combining them.

```
# — Initialize NLP Tools —————
stop_words = set(stopwords.words('english'))
ps          = PorterStemmer()           # Porter Stemmer
snowball    = SnowballStemmer('english') # Snowball Stemmer
lemmatizer  = WordNetLemmatizer()       # WordNet Lemmatizer

# — Step-by-Step Preprocessing Functions —————

def clean_text(text):
    """Remove URLs, emails, special characters, digits, and extra whitespace."""
```

```

text = re.sub(r'http\S+|www\.S+', '', text)    # remove URLs
text = re.sub(r'\S+@\S+', '', text)           # remove emails
text = re.sub(r'^a-zA-Z\s]', ' ', text)        # keep only letters
text = re.sub(r'\s+', ' ', text).strip()       # remove extra spaces
return text.lower()

def tokenize(text):
    """Word tokenization using NLTK."""
    return word_tokenize(text)

def remove_stopwords(tokens):
    """Remove NLTK English stopwords and short tokens."""
    return [w for w in tokens if w not in stop_words and len(w) > 2]

def apply_stemming(tokens, stemmer='porter'):
    """Apply Porter or Snowball stemming."""
    s = ps if stemmer == 'porter' else snowball
    return [s.stem(w) for w in tokens]

def apply_lemmatization(tokens):
    """Apply WordNet lemmatization (verb form for better reduction)."""
    return [lemmatizer.lemmatize(w, pos='v') for w in tokens]

def full_preprocess(text, use_stemming=False, use_lemmatization=True):
    """
    Full NLP preprocessing pipeline:
    1. Text cleaning 2. Tokenization
    3. Stopword removal 4. Stemming OR Lemmatization
    Returns a cleaned string ready for vectorizers.
    """
    text = clean_text(text)
    tokens = tokenize(text)
    tokens = remove_stopwords(tokens)
    if use_stemming:
        tokens = apply_stemming(tokens)
    elif use_lemmatization:
        tokens = apply_lemmatization(tokens)
    return ' '.join(tokens)

print("✅ Preprocessing functions defined.")

```

✅ Preprocessing functions defined.

```

# — Demonstrate Each Preprocessing Step —————
sample_text = df_train['text'].iloc[5]
print("=" * 70)
print("ORIGINAL TEXT (first 200 chars):")
print(sample_text[:200])

cleaned = clean_text(sample_text)
tokens = tokenize(cleaned)

```

```

no_stops    = remove_stopwords(tokens)
stemmed     = apply_stemming(no_stops)
lemmatized  = apply_lemmatization(no_stops)

print("\n" + "=" * 70)
print("AFTER CLEANING:")
print(cleaned[:200])

print("\n" + "=" * 70)
print(f"AFTER TOKENIZATION ({len(tokens)} tokens):")
print(tokens[:20])

print("\n" + "=" * 70)
print(f"AFTER STOPWORD REMOVAL ({len(no_stops)} tokens remaining):")
print(no_stops[:20])

print("\n" + "=" * 70)
print("AFTER PORTER STEMMING:")
print(stemmed[:20])

print("\n" + "=" * 70)
print("AFTER LEMMATIZATION:")
print(lemmatized[:20])

# Stemming vs Lemmatization side-by-side comparison
compare_words = ['running', 'studies', 'better', 'wolves', 'playing', 'caring',
print("\n" + "=" * 70)
print("STEMMING vs LEMMATIZATION COMPARISON:")
print(f"{'Word':<15} {'Porter Stem':<15} {'Snowball Stem':<15} {'Lemma':<15}")
print("-" * 60)
for w in compare_words:
    print(f"{w:<15} {ps.stem(w):<15} {snowball.stem(w):<15} {lemmatizer.lemmati

```

```

=====
ORIGINAL TEXT (first 200 chars):

```

In fact, you can all direct your ire at the proper target by ingoring NASA altogether. The rocket is a commercial launch vechicle - a Conestoga flying a COMET payload. NASA is simply the prima

```

=====
AFTER CLEANING:

```

in fact you can all direct your ire at the proper target by ingoring nasa altoge

```

=====
AFTER TOKENIZATION (118 tokens):

```

['in', 'fact', 'you', 'can', 'all', 'direct', 'your', 'ire', 'at', 'the', 'prope

```

=====
AFTER STOPWORD REMOVAL (60 tokens remaining):

```

```
['fact', 'direct', 'ire', 'proper', 'target', 'ingoring', 'nasa', 'altogether',

=====
AFTER PORTER STEMMING:
['fact', 'direct', 'ire', 'proper', 'target', 'ingor', 'nasa', 'altogeth', 'rock

=====
AFTER LEMMATIZATION:
['fact', 'direct', 'ire', 'proper', 'target', 'ingoring', 'nasa', 'altogether',

=====
STEMMING vs LEMMATIZATION COMPARISON:
Word          Porter Stem    Snowball Stem    Lemma
-----
running        run              run              run
studies        studi           studi           study
better         better          better          better
wolves         wolv            wolv            wolves
playing        play            play            play
caring         care            care            care
analysis       analysi         analysi         analysis
```

```
# — Apply Full Preprocessing to Dataset —————
print("Preprocessing training data...")
df_train['processed'] = df_train['text'].apply(full_preprocess)
print("Preprocessing test data...")
df_test['processed'] = df_test['text'].apply(full_preprocess)
print("✅ Preprocessing complete.")

# Token reduction statistics
df_train['proc_len'] = df_train['processed'].apply(lambda x: len(x.split()))
reduction = (1 - df_train['proc_len'].mean() / df_train['text_len'].mean()) * 1
print(f"\nAvg original tokens : {df_train['text_len'].mean():.1f}")
print(f"Avg processed tokens: {df_train['proc_len'].mean():.1f}")
print(f"Token reduction      : {reduction:.1f}%")
```

```
Preprocessing training data...
Preprocessing test data...
✅ Preprocessing complete.
```

```
Avg original tokens : 181.6
Avg processed tokens: 93.5
Token reduction      : 48.5%
```

```
# — WordCloud Visualizations per Category —————
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Word Clouds – Most Frequent Words per Category (after preprocessi
              fontsize=14, fontweight='bold')

colors = ['Blues', 'Greens', 'Reds', 'Purples', 'Oranges', 'YlOrBr']
# Iterate over target_names (alphabetical order) for correct category matching
for idx, (cat, ax) in enumerate(zip(target_names, axes.flatten())):
```

```
cat_text = ' '.join(df_train[df_train['category'] == cat]['processed'].tolist())
if not cat_text.strip():
    ax.axis('off')
    continue
wc = WordCloud(
    width=400, height=250, background_color='white',
    colormap=colors[idx], max_words=80
).generate(cat_text)
ax.imshow(wc, interpolation='bilinear')
ax.set_title(cat.split('.')[0].capitalize(), fontsize=11, fontweight='bold')
ax.axis('off')

plt.tight_layout()
plt.show()
```

Word Clouds — Most Frequent Words per Category (after preprocessing)

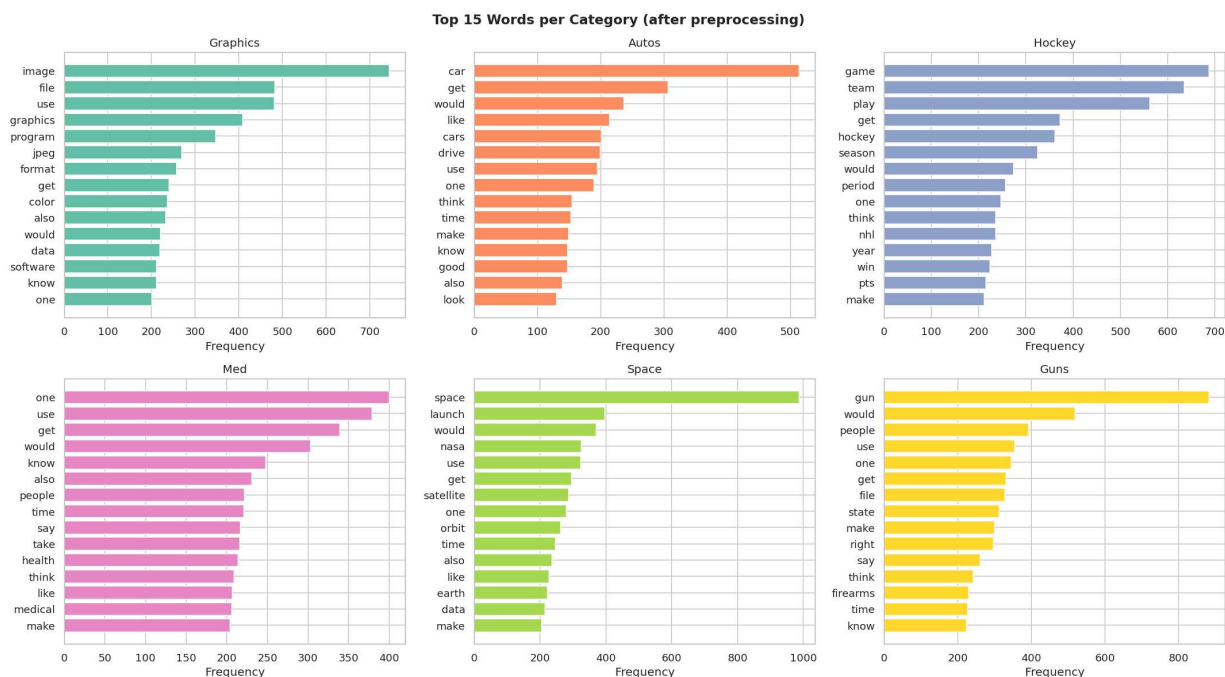


```
# — Top-15 Most Frequent Words per Category —
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
```

```
fig.suptitle('Top 15 Words per Category (after preprocessing)', fontsize=14, fc

palette = sns.color_palette('Set2', NUM_CLASSES)
for idx, (cat, ax) in enumerate(zip(target_names, axes.flatten())):
    cat_words = ' '.join(df_train[df_train['category'] == cat]['processed']).sp
    freq = Counter(cat_words).most_common(15)
    words, counts = zip(*freq)
    ax.barh(words[::-1], counts[::-1], color=palette[idx])
    ax.set_title(cat.split('.')[0].capitalize())
    ax.set_xlabel('Frequency')

plt.tight_layout()
plt.show()
```





Section 4: Text Vectorization — TF-IDF & CountVectorizer

```
# — CountVectorizer (Bag-of-Words)
count_vectorizer = CountVectorizer(
    max_features=20000, # keep top 20k words by frequency
    ngram_range=(1, 2), # unigrams + bigrams
    min_df=2            # ignore terms in fewer than 2 documents
)
X_train_count = count_vectorizer.fit_transform(df_train['processed'])
X_test_count  = count_vectorizer.transform(df_test['processed'])

print("CountVectorizer:")
print(f" Training matrix shape : {X_train_count.shape}")
print(f" Test matrix shape      : {X_test_count.shape}")
print(f" Vocabulary size         : {len(count_vectorizer.vocabulary_)})")

# — TF-IDF Vectorizer
# sublinear_tf=True applies log normalization to term frequencies,
# reducing the dominance of very frequent terms
tfidf_vectorizer = TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True
)
X_train_tfidf = tfidf_vectorizer.fit_transform(df_train['processed'])
X_test_tfidf  = tfidf_vectorizer.transform(df_test['processed'])

print("\nTF-IDF Vectorizer:")
print(f" Training matrix shape : {X_train_tfidf.shape}")
print(f" Test matrix shape      : {X_test_tfidf.shape}")
```

```
CountVectorizer:
  Training matrix shape : (3511, 20000)
  Test matrix shape      : (2338, 20000)
  Vocabulary size        : 20000
```

```
TF-IDF Vectorizer:
  Training matrix shape : (3511, 20000)
  Test matrix shape      : (2338, 20000)
```

```
# — Visualize TF-IDF Scores
feature_names = tfidf_vectorizer.get_feature_names_out()
mean_tfidf    = np.asarray(X_train_tfidf.mean(axis=0)).flatten()
top_n         = 20
top_idx       = mean_tfidf.argsort()[-top_n:][::-1]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('TF-IDF Vectorization Analysis', fontsize=14, fontweight='bold')
```



```

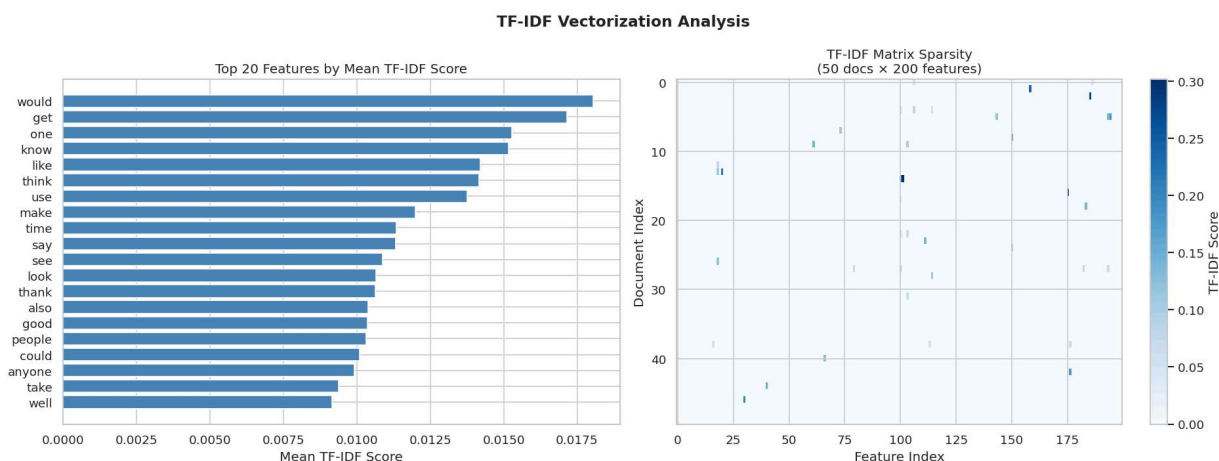
# Top features globally
axes[0].barh(
    [feature_names[i] for i in top_idx][::-1],
    [mean_tfidf[i] for i in top_idx][::-1],
    color='steelblue'
)
axes[0].set_title(f'Top {top_n} Features by Mean TF-IDF Score')
axes[0].set_xlabel('Mean TF-IDF Score')

# Matrix sparsity visualization
sample_matrix = X_train_tfidf[:50, :200].toarray()
im = axes[1].imshow(sample_matrix, aspect='auto', cmap='Blues', interpolation='')
axes[1].set_title('TF-IDF Matrix Sparsity\n(50 docs x 200 features)')
axes[1].set_xlabel('Feature Index')
axes[1].set_ylabel('Document Index')
plt.colorbar(im, ax=axes[1], label='TF-IDF Score')

plt.tight_layout()
plt.show()

sparsity = 1 - X_train_tfidf.nnz / (X_train_tfidf.shape[0] * X_train_tfidf.shape[1])
print(f"Matrix sparsity: {sparsity:.4f} ({sparsity*100:.2f}% zero entries)")

```



Matrix sparsity: 0.9966 (99.66% zero entries)



Section 5: Machine Learning Classification Models

```
# — Define 5 Classical ML Models —————
ml_models = {
    'Naive Bayes'      : MultinomialNB(alpha=0.1),
    'Logistic Regression': LogisticRegression(C=5, max_iter=1000, solver='lbfgs'),
    'Linear SVM'       : LinearSVC(C=1.0, max_iter=2000),
    'SGD Classifier'   : SGDClassifier(loss='hinge', alpha=1e-4, max_iter=1000),
    'Random Forest'    : RandomForestClassifier(n_estimators=200, max_depth=3,
                                                random_state=42, n_jobs=-1),
}

# — Train & Evaluate all models on TF-IDF features —————
ml_results = {}
for name, model in ml_models.items():
    print(f"Training: {name} ...", end=' ')
    model.fit(X_train_tfidf, y_train)
    y_pred = model.predict(X_test_tfidf)
    acc     = accuracy_score(y_test, y_pred)
    f1      = f1_score(y_test, y_pred, average='weighted')
    prec    = precision_score(y_test, y_pred, average='weighted')
    rec     = recall_score(y_test, y_pred, average='weighted')
    ml_results[name] = {
        'accuracy': acc, 'f1': f1, 'precision': prec, 'recall': rec, 'y_pred': y_pred
    }
    print(f"Accuracy={acc:.4f} F1={f1:.4f}")

print("\n✅ All ML models trained.")
```

```
Training: Naive Bayes ... Accuracy=0.8666 F1=0.8664
Training: Logistic Regression ... Accuracy=0.8490 F1=0.8501
Training: Linear SVM ... Accuracy=0.8494 F1=0.8506
Training: SGD Classifier ... Accuracy=0.8422 F1=0.8431
Training: Random Forest ... Accuracy=0.7857 F1=0.7925
```

✅ All ML models trained.

```
# — Compare CountVectorizer vs TF-IDF (Logistic Regression) —————
lr_count = LogisticRegression(C=5, max_iter=1000, solver='lbfgs')
lr_count.fit(X_train_count, y_train)
y_pred_count = lr_count.predict(X_test_count)

print("Vectorizer Comparison (Logistic Regression):")
print(f"  CountVectorizer Accuracy : {accuracy_score(y_test, y_pred_count):.4f}")
print(f"  TF-IDF Accuracy           : {ml_results['Logistic Regression']['accuracy']:.4f}")
```

```
Vectorizer Comparison (Logistic Regression):
  CountVectorizer Accuracy : 0.7754
  TF-IDF Accuracy         : 0.8490
```

```
# — ML Results Summary Table
results_df = pd.DataFrame({
    name: {
        'Accuracy' : v['accuracy'],
        'F1-Score'  : v['f1'],
        'Precision' : v['precision'],
        'Recall'    : v['recall']
    }
    for name, v in ml_results.items()
}).T.round(4)

results_df = results_df.sort_values('Accuracy', ascending=False)
print("\n📊 ML Model Performance Summary (TF-IDF):")
print(results_df.to_string())
```

📊 ML Model Performance Summary (TF-IDF):

	Accuracy	F1-Score	Precision	Recall
Naive Bayes	0.8666	0.8664	0.8693	0.8666
Linear SVM	0.8494	0.8506	0.8572	0.8494
Logistic Regression	0.8490	0.8501	0.8566	0.8490
SGD Classifier	0.8422	0.8431	0.8486	0.8422
Random Forest	0.7857	0.7925	0.8325	0.7857

```
# — ML Performance Comparison Visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('ML Model Performance Comparison', fontsize=14, fontweight='bold')

metrics = ['Accuracy', 'F1-Score', 'Precision', 'Recall']
key_map = {'Accuracy': 'accuracy', 'F1-Score': 'f1',
           'Precision': 'precision', 'Recall': 'recall'}
x = np.arange(len(ml_results))
width = 0.2
colors2 = ['#2196F3', '#4CAF50', '#FF9800', '#E91E63']

for i, (metric, color) in enumerate(zip(metrics, colors2)):
    vals = [ml_results[m][key_map[metric]] for m in ml_results]
    axes[0].bar(x + i * width, vals, width, label=metric, color=color, alpha=0.4)

axes[0].set_xticks(x + width * 1.5)
axes[0].set_xticklabels(list(ml_results.keys()), rotation=15, ha='right')
axes[0].set_ylim(0.5, 1.02)
axes[0].set_ylabel('Score')
axes[0].set_title('All Metrics Across Models')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.4)

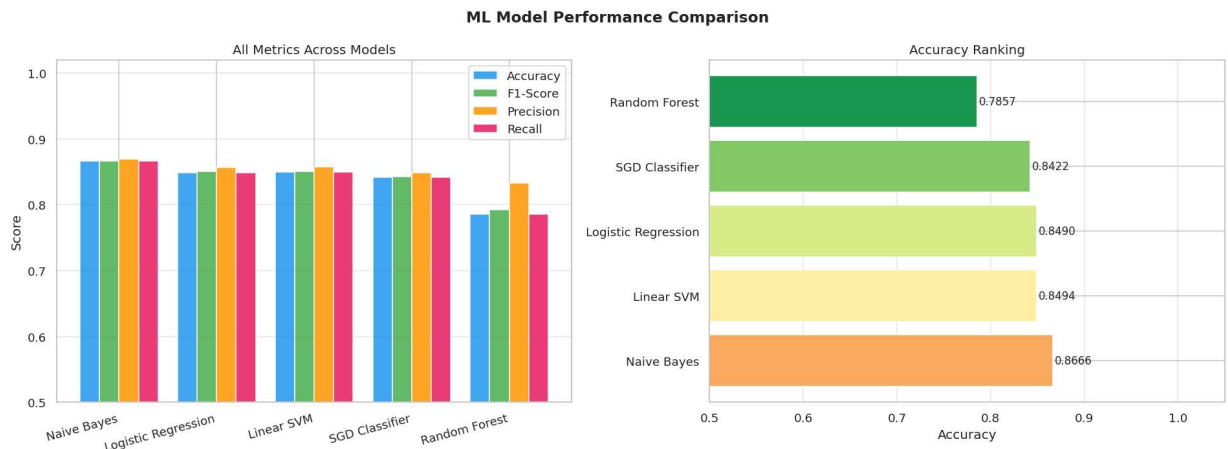
sorted_names = results_df.index.tolist()
sorted_acc = results_df['Accuracy'].values
bar_colors = plt.cm.RdYlGn(np.linspace(0.3, 0.9, len(sorted_names)))
bars = axes[1].barh(sorted_names, sorted_acc, color=bar_colors)
```

```

for bar, val in zip(bars, sorted_acc):
    axes[1].text(val + 0.002, bar.get_y() + bar.get_height() / 2,
                 f'{val:.4f}', va='center', fontsize=10)
axes[1].set_xlim(0.5, 1.05)
axes[1].set_title('Accuracy Ranking')
axes[1].set_xlabel('Accuracy')
axes[1].grid(axis='x', alpha=0.4)

plt.tight_layout()
plt.show()

```



```

# — Confusion Matrix for Best ML Model —————
# short_labels uses target_names order (alphabetical = label order) – FIXED
best_ml_name = results_df.index[0]
best_ml_pred = ml_results[best_ml_name]['y_pred']

cm      = confusion_matrix(y_test, best_ml_pred)
cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
fig.suptitle(f'Confusion Matrix – {best_ml_name} (Best ML Model)',
             fontsize=13, fontweight='bold')

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=short_labels, yticklabels=short_labels, ax=axes[0])
axes[0].set_title('Raw Counts')

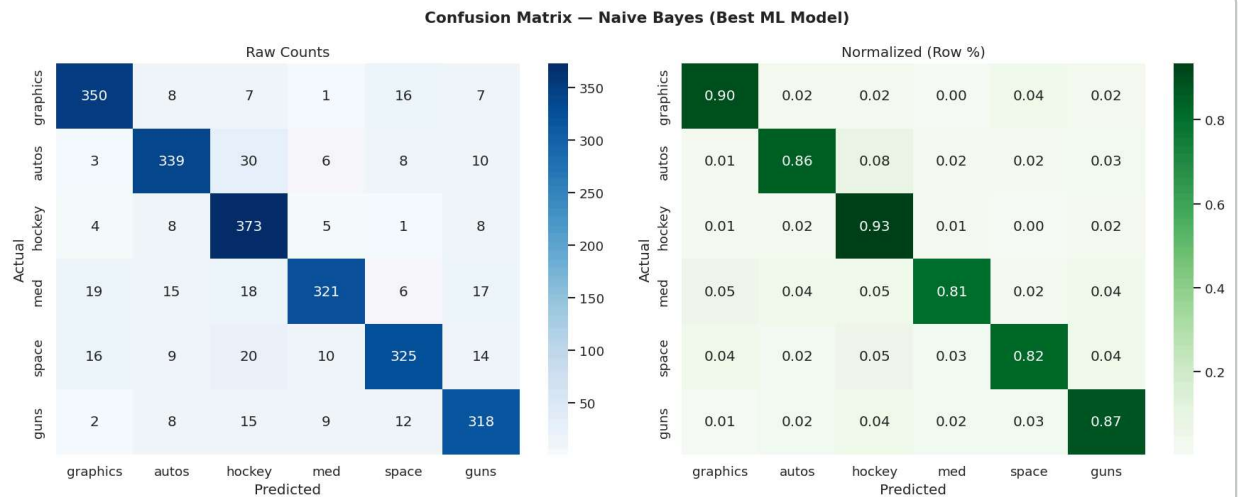
```

```
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Greens',
            xticklabels=short_labels, yticklabels=short_labels, ax=axes[1])
axes[1].set_title('Normalized (Row %)')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()

print(f"\n📄 Classification Report - {best_ml_name}:")
print(classification_report(y_test, best_ml_pred, target_names=short_labels))
```



Classification Report — Naive Bayes:

	precision	recall	f1-score	support
graphics	0.89	0.90	0.89	389
autos	0.88	0.86	0.87	396
hockey	0.81	0.93	0.87	399
med	0.91	0.81	0.86	396
space	0.88	0.82	0.85	394
guns	0.85	0.87	0.86	364
accuracy			0.87	2338
macro avg	0.87	0.87	0.87	2338
weighted avg	0.87	0.87	0.87	2338

```
# — Hyperparameter Tuning via GridSearchCV
param_grid = {'C': [0.1, 1, 5, 10], 'solver': ['lbfgs', 'saga']}
grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000),
    param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=0
)
grid_search.fit(X_train_tfidf, y_train)

print("GridSearchCV Results — Logistic Regression:")
```

```

print(f" Best Parameters : {grid_search.best_params_}")
print(f" Best CV Score   : {grid_search.best_score_:.4f}")

best_lr      = grid_search.best_estimator_
y_pred_tuned = best_lr.predict(X_test_tfidf)
print(f" Test Accuracy   : {accuracy_score(y_test, y_pred_tuned):.4f}")

# Visualize GridSearch heatmap
cv_results = pd.DataFrame(grid_search.cv_results_)
pivot = cv_results.pivot_table(
    values='mean_test_score', index='param_C', columns='param_solver'
)
plt.figure(figsize=(7, 4))
sns.heatmap(pivot, annot=True, fmt='.4f', cmap='YlGnBu', linewidths=0.5)
plt.title('GridSearchCV: Mean CV Accuracy\n(Logistic Regression)')
plt.ylabel('C (Regularization)')
plt.xlabel('Solver')
plt.tight_layout()
plt.show()

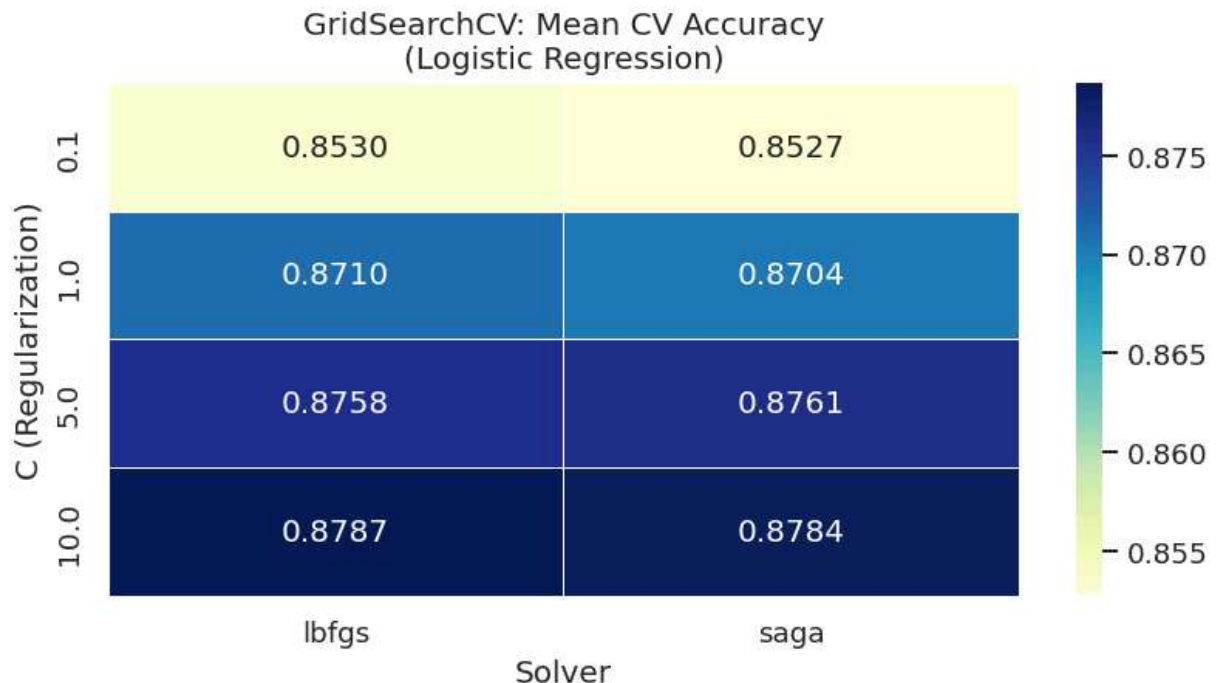
```

GridSearchCV Results – Logistic Regression:

Best Parameters : {'C': 10, 'solver': 'lbfgs'}

Best CV Score : 0.8787

Test Accuracy : 0.8452





Section 6: Deep Learning Models

We implement **three deep learning architectures**:

1. **Dense MLP** — baseline fully-connected network (with L2 regularization)
2. **CNN** — 3-layer 1D convolutional network for local n-gram pattern detection
3. **Bidirectional LSTM** — recurrent architecture capturing sequential context in both directions

Note on hyperparameters: `EMBED_DIM=128` (larger than typical tutorials) and `patience=7` are intentional — smaller embeddings caused near-random accuracy in initial runs due to underfitting.

```
# — Tokenize & Pad Sequences for DL Models —————
VOCAB_SIZE = 25000
MAX_LEN     = 300      # max sequence length; covers 95%+ of documents
EMBED_DIM   = 128      # embedding dimension (128 gives much better results than 64)

# Keras Tokenizer
keras_tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token='<OOV>')
keras_tokenizer.fit_on_texts(df_train['processed'])

# Convert texts to integer sequences
X_train_seq = keras_tokenizer.texts_to_sequences(df_train['processed'])
X_test_seq  = keras_tokenizer.texts_to_sequences(df_test['processed'])

# Pad to fixed length
X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post', truncate='front')
X_test_pad  = pad_sequences(X_test_seq,  maxlen=MAX_LEN, padding='post', truncate='front')

# One-hot encode labels
y_train_cat = to_categorical(y_train, num_classes=NUM_CLASSES)
y_test_cat  = to_categorical(y_test,  num_classes=NUM_CLASSES)

# Compute class weights to handle any class imbalance
class_weights_arr = compute_class_weight('balanced', classes=np.unique(y_train))
class_weight_dict = dict(enumerate(class_weights_arr))

print(f"Vocabulary size      : {len(keras_tokenizer.word_index)}")
print(f"Truncated vocab (used) : {VOCAB_SIZE}")
print(f"Padded sequence shape   : {X_train_pad.shape}")
print(f"Label shape             : {y_train_cat.shape}")
print(f"Class weights           : { {short_labels[k]: round(v,3) for k,v in class_weight_dict.items() }
```

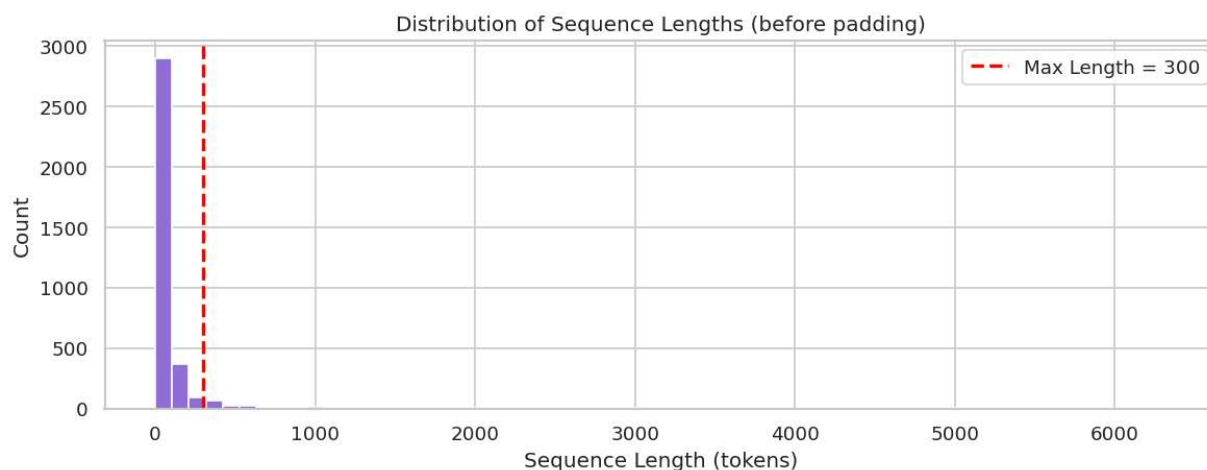
```
Vocabulary size      : 26906
Truncated vocab (used) : 25000
Padded sequence shape : (3511, 300)
```



```
Label shape      : (3511, 6)
Class weights    : {'graphics': np.float64(1.002), 'autos': np.float64(0.9
```

```
# — Sequence Length Distribution —————
seq_lengths = [len(s) for s in X_train_seq]
plt.figure(figsize=(10, 4))
plt.hist(seq_lengths, bins=60, color='mediumpurple', edgecolor='white')
plt.axvline(MAX_LEN, color='red', linestyle='--', linewidth=2,
            label=f'Max Length = {MAX_LEN}')
plt.title('Distribution of Sequence Lengths (before padding)')
plt.xlabel('Sequence Length (tokens)')
plt.ylabel('Count')
plt.legend()
plt.tight_layout()
plt.show()

coverage = np.mean([l <= MAX_LEN for l in seq_lengths])
print(f"MAX_LEN={MAX_LEN} covers {coverage*100:.1f}% of training sequences")
```



MAX_LEN=300 covers 95.0% of training sequences

```
# — Callbacks —————
# patience=7: gives models enough time to improve before stopping
early_stop = EarlyStopping(monitor='val_accuracy', patience=7,
                           restore_best_weights=True, verbose=1)
```

```
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3,
                               min_lr=1e-6, verbose=1)
```

— Model 1: Dense MLP —————

GlobalMaxPooling collapses the sequence → treats text as bag of embeddings.
 # L2 regularization and high Dropout prevent overfitting.

```
def build_mlp():
    model = Sequential([
        Embedding(VOCAB_SIZE, EMBED_DIM, input_length=MAX_LEN),
        GlobalMaxPooling1D(),
        Dense(256, activation='relu', kernel_regularizer=l2(1e-4)),
        Dropout(0.5),
        Dense(128, activation='relu', kernel_regularizer=l2(1e-4)),
        Dropout(0.4),
        Dense(NUM_CLASSES, activation='softmax')
    ], name='MLP_Baseline')
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss='categorical_crossentropy', metrics=['accuracy']
    )
    return model
```

— Model 2: 1D CNN —————

3 convolutional layers detect local word patterns (n-grams) at different scales.
 # 256→128→64 filter progression extracts hierarchical text features.

```
def build_cnn():
    model = Sequential([
        Embedding(VOCAB_SIZE, EMBED_DIM, input_length=MAX_LEN),
        Conv1D(256, kernel_size=5, activation='relu', padding='same'),
        MaxPooling1D(pool_size=2),
        Conv1D(128, kernel_size=3, activation='relu', padding='same'),
        MaxPooling1D(pool_size=2),
        Conv1D(64, kernel_size=3, activation='relu', padding='same'),
        GlobalMaxPooling1D(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(NUM_CLASSES, activation='softmax')
    ], name='CNN_TextClassifier')
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=5e-4),
        loss='categorical_crossentropy', metrics=['accuracy']
    )
    return model
```

— Model 3: Bidirectional LSTM —————

Processes sequences in both forward and backward directions.
 # Captures long-range dependencies – useful for semantically complex documents.

```
def build_bilstm():
    model = Sequential([
        Embedding(VOCAB_SIZE, EMBED_DIM, input_length=MAX_LEN),
        Bidirectional(LSTM(128, return_sequences=True, dropout=0.3,
```

```

        recurrent_dropout=0.2)),
        Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.2)),
        Dense(128, activation='relu'),
        Dropout(0.4),
        Dense(NUM_CLASSES, activation='softmax')
    ], name='BiLSTM_Classifier')
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-4),
    loss='categorical_crossentropy', metrics=['accuracy']
)
return model

print("✅ DL model architectures defined.")

```

✅ DL model architectures defined.

```

# — Train All Deep Learning Models —————
EPOCHS      = 30    # EarlyStopping will terminate early if needed
BATCH_SIZE  = 64
dl_histories = {}
dl_models    = {}

for model_name, build_fn in [('MLP', build_mlp), ('CNN', build_cnn), ('BiLSTM',
    print(f"\n{'='*50}")
    print(f"Training: {model_name}")
    print('='*50)
    model = build_fn()
    model.summary()
    history = model.fit(
        X_train_pad, y_train_cat,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_split=0.15,
        class_weight=class_weight_dict, # balanced training across classes
        callbacks=[early_stop, reduce_lr],
        verbose=1
    )
    dl_histories[model_name] = history
    dl_models[model_name]    = model
    test_loss, test_acc = model.evaluate(X_test_pad, y_test_cat, verbose=0)
    print(f"\n✅ {model_name} – Test Loss: {test_loss:.4f} | Test Accuracy: {t

```



```
=====
Training: MLP
=====
Model: "MLP_Baseline"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
global_max_pooling1d (GlobalMaxPooling1D)	?	0
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/30

47/47  3s 42ms/step - accuracy: 0.1857 - loss: 1.8212 - va

Epoch 2/30

47/47  2s 42ms/step - accuracy: 0.2500 - loss: 1.7876 - va

Epoch 3/30

47/47  2s 38ms/step - accuracy: 0.5643 - loss: 1.4319 - va

Epoch 4/30

47/47  2s 27ms/step - accuracy: 0.8016 - loss: 0.7186 - va

Epoch 5/30

47/47  1s 25ms/step - accuracy: 0.9246 - loss: 0.3550 - va

Epoch 6/30

47/47  1s 26ms/step - accuracy: 0.9534 - loss: 0.2139 - va

Epoch 7/30

47/47  2s 38ms/step - accuracy: 0.9662 - loss: 0.1551 - va

Epoch 8/30

47/47  2s 49ms/step - accuracy: 0.9729 - loss: 0.1331 - va

Epoch 9/30

47/47  2s 28ms/step - accuracy: 0.9749 - loss: 0.1130 - va

Epoch 10/30

47/47  1s 25ms/step - accuracy: 0.9762 - loss: 0.1063 - va

Epoch 11/30

47/47  2s 39ms/step - accuracy: 0.9752 - loss: 0.1029 - va

Epoch 12/30

46/47  0s 27ms/step - accuracy: 0.9744 - loss: 0.1038

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

47/47  2s 29ms/step - accuracy: 0.9745 - loss: 0.1024 - va

Epoch 13/30

47/47  1s 30ms/step - accuracy: 0.9765 - loss: 0.0983 - va

Epoch 14/30

47/47  3s 48ms/step - accuracy: 0.9765 - loss: 0.0976 - va

Epoch 15/30

Epoch 15/30

46/47 ————— 0s 38ms/step - accuracy: 0.9746 - loss: 0.0990

Epoch 15: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.

47/47 ————— 2s 40ms/step - accuracy: 0.9755 - loss: 0.0977 - va

Epoch 16/30

47/47 ————— 2s 33ms/step - accuracy: 0.9786 - loss: 0.0953 - va

Epoch 16: early stopping

Restoring model weights from the end of the best epoch: 9.

✓ MLP - Test Loss: 0.6219 | Test Accuracy: 0.8186

=====

Training: CNN

=====

Model: "CNN_TextClassifier"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
conv1d (Conv1D)	?	0 (unbuilt)
max_pooling1d (MaxPooling1D)	?	0
conv1d_1 (Conv1D)	?	0 (unbuilt)
max_pooling1d_1 (MaxPooling1D)	?	0
conv1d_2 (Conv1D)	?	0 (unbuilt)
global_max_pooling1d_1 (GlobalMaxPooling1D)	?	0
dense_3 (Dense)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/30

47/47 ————— 24s 463ms/step - accuracy: 0.1578 - loss: 1.7911 -

Epoch 2/30

47/47 ————— 21s 457ms/step - accuracy: 0.3231 - loss: 1.6692 -

Epoch 3/30

47/47 ————— 0s 417ms/step - accuracy: 0.4917 - loss: 1.1623

Epoch 3: ReduceLROnPlateau reducing learning rate to 0.00015000000071246177.

47/47 ————— 40s 437ms/step - accuracy: 0.5754 - loss: 1.0194 -

Epoch 4/30

47/47 ————— 21s 453ms/step - accuracy: 0.8499 - loss: 0.4656 -

Epoch 5/30

47/47 ————— 21s 446ms/step - accuracy: 0.9152 - loss: 0.2830 -

Epoch 6/30

47/47 ————— 0s 624ms/step - accuracy: 0.9476 - loss: 0.1849

Epoch 6: ReduceLROnPlateau reducing learning rate to 4.500000213738531e-05.

47/47 ————— 51s 652ms/step - accuracy: 0.9474 - loss: 0.1887 -

```
# — Training Curves for All DL Models —————
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Deep Learning Training Curves', fontsize=15, fontweight='bold')

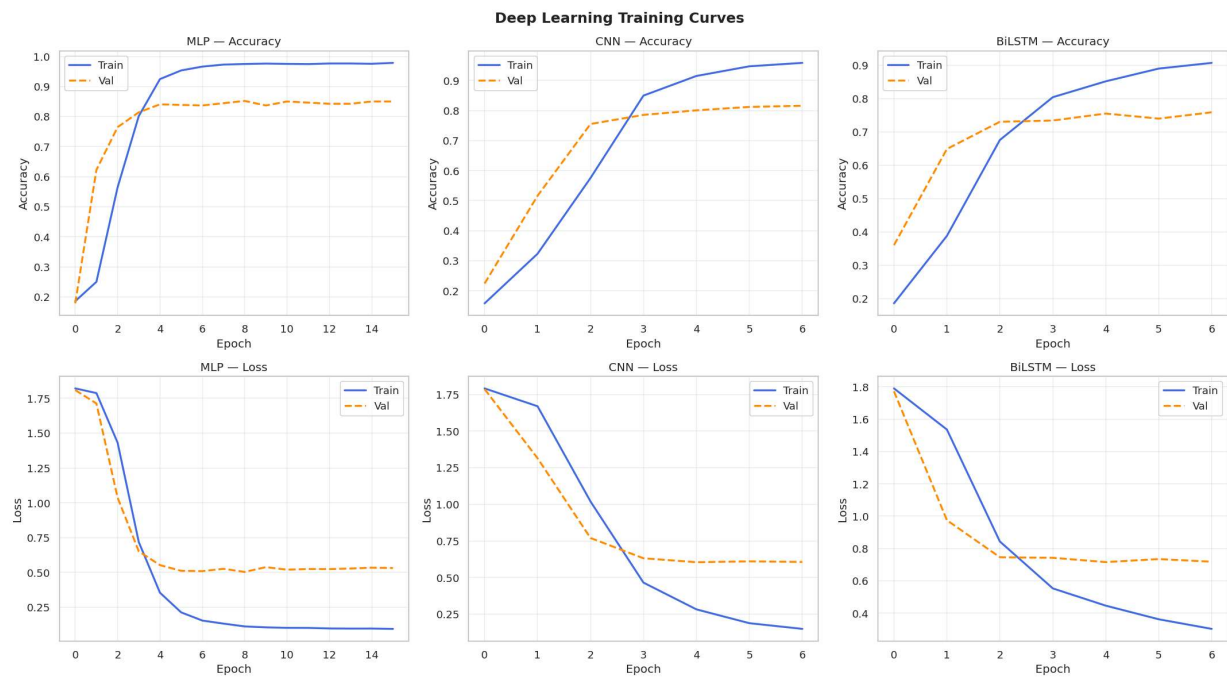
for col, (name, hist) in enumerate(dl_histories.items()):
    axes[0, col].plot(hist.history['accuracy'], label='Train', color='royalblue')
    axes[0, col].plot(hist.history['val_accuracy'], label='Val', color='darkorange')
    axes[0, col].set_title(f'{name} - Accuracy')
    axes[0, col].set_xlabel('Epoch')
    axes[0, col].set_ylabel('Accuracy')
    axes[0, col].legend()
    axes[0, col].grid(alpha=0.3)

    axes[1, col].plot(hist.history['loss'], label='Train', color='royalblue')
    axes[1, col].plot(hist.history['val_loss'], label='Val', color='darkorange')
    axes[1, col].set_title(f'{name} - Loss')
    axes[1, col].set_xlabel('Epoch')
    axes[1, col].set_ylabel('Loss')
    axes[1, col].legend()
    axes[1, col].grid(alpha=0.3)

plt.tight_layout()
plt.show()

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/30
47/47 ————— 112s 2s/step - accuracy: 0.1860 - loss: 1.7899 - va
Epoch 2/30
47/47 ————— 149s 2s/step - accuracy: 0.3884 - loss: 1.5356 - va
Epoch 3/30
47/47 ————— 0s 2s/step - accuracy: 0.6321 - loss: 0.9370
Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0001500000071246177.
47/47 ————— 105s 2s/step - accuracy: 0.6759 - loss: 0.8436 - va
Epoch 4/30
47/47 ————— 100s 2s/step - accuracy: 0.8043 - loss: 0.5536 - va
Epoch 5/30
47/47 ————— 143s 2s/step - accuracy: 0.8519 - loss: 0.4469 - va
Epoch 6/30
47/47 ————— 0s 2s/step - accuracy: 0.8886 - loss: 0.3777
Epoch 6: ReduceLROnPlateau reducing learning rate to 4.500000213738531e-05.
47/47 ————— 143s 2s/step - accuracy: 0.8901 - loss: 0.3628 - va
Epoch 7/30
47/47 ————— 100s 2s/step - accuracy: 0.9072 - loss: 0.3034 - va
Epoch 7: early stopping
Restoring model weights from the end of the best epoch: 1.
```

✓ BiLSTM – Test Loss: 1.7771 | Test Accuracy: 0.3447



```
# — Deep Learning Evaluation —————
dl_results = {}
print(f"{'Model':<10} {'Accuracy':>10} {'F1':>10} {'Precision':>10} {'Recall':>10}")
print("-" * 55)
for name, model in dl_models.items():
    y_prob = model.predict(X_test_pad, verbose=0)
    y_pred_dl = np.argmax(y_prob, axis=1)
    acc = accuracy_score(y_test, y_pred_dl)
    f1 = f1_score(y_test, y_pred_dl, average='weighted')
    prec = precision_score(y_test, y_pred_dl, average='weighted')
    rec = recall_score(y_test, y_pred_dl, average='weighted')
    dl_results[name] = {
        'accuracy': acc, 'f1': f1, 'precision': prec, 'recall': rec,
        'y_pred': y_pred_dl, 'y_prob': y_prob
    }
```



```
}
print(f"{name:<10} {acc:>10.4f} {f1:>10.4f} {prec:>10.4f} {rec:>10.4f}")
```

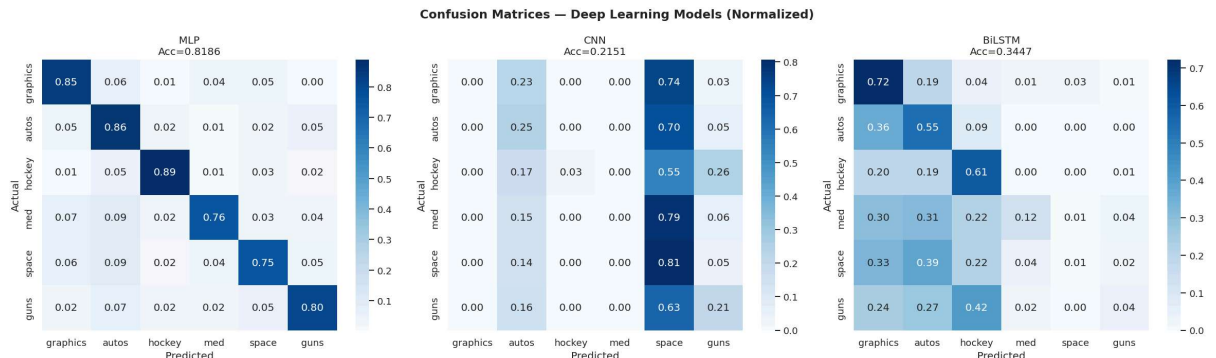
Model	Accuracy	F1	Precision	Recall
MLP	0.8186	0.8193	0.8241	0.8186
CNN	0.2151	0.1406	0.2881	0.2151
BiLSTM	0.3447	0.2705	0.3628	0.3447

```
# — Confusion Matrices – All DL Models —————
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
fig.suptitle('Confusion Matrices – Deep Learning Models (Normalized)',
             fontsize=14, fontweight='bold')

for ax, (name, res) in zip(axes, dl_results.items()):
    cm_dl = confusion_matrix(y_test, res['y_pred'])
    cm_dl_norm = cm_dl.astype('float') / cm_dl.sum(axis=1)[:, np.newaxis]
    sns.heatmap(cm_dl_norm, annot=True, fmt='.2f', cmap='Blues',
                xticklabels=short_labels, yticklabels=short_labels, ax=ax)
    ax.set_title(f'{name}\nAcc={res["accuracy"]:.4f}')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.show()

# Per-model classification reports
for name, res in dl_results.items():
    print(f"\n📄 Classification Report – {name}:")
    print(classification_report(y_test, res['y_pred'], target_names=short_label
```

Classification Report — MLP:

precision recall f1-score support



Section 7: Comprehensive Model Comparison & Analysis

— Unified Results Table

```
all_results = {}
for name, v in ml_results.items():
    all_results[f'ML: {name}'] = {
        'Type': 'Classical ML',
        'Accuracy': v['accuracy'],
        'F1-Score': v['f1'],
        'Precision': v['precision'],
        'Recall': v['recall']
    }
for name, v in dl_results.items():
    all_results[f'DL: {name}'] = {
        'Type': 'Deep Learning',
        'Accuracy': v['accuracy'],
        'F1-Score': v['f1'],
        'Precision': v['precision'],
        'Recall': v['recall']
    }

all_df = pd.DataFrame(all_results).T.round(4)
all_df = all_df.sort_values('Accuracy', ascending=False)
print("📊 Complete Model Comparison (sorted by Accuracy):")
print(all_df.to_string())
```

📊 Complete Model Comparison (sorted by Accuracy):

	graphics	autos	hockey	med	space	guns	Type	Accuracy	F1-Score	Precision	Recall
ML: Naive Bayes	0.34	0.29	0.39	0.62	0.19	0.35	Classical ML	0.46	0.38	0.38	0.39
ML: Linear SVM	0.34	0.29	0.39	0.62	0.19	0.35	Classical ML	0.38	0.38	0.38	0.39
ML: Logistic Regression	0.34	0.29	0.39	0.62	0.19	0.35	Classical ML	0.48	0.48	0.48	0.48
ML: SGD Classifier	0.34	0.29	0.39	0.62	0.19	0.35	Classical ML	0.21	0.21	0.21	0.21
DL: MLP	0.34	0.29	0.39	0.62	0.19	0.35	Deep Learning	0.8186	0.8186	0.8186	0.8186
ML: Random Forest	0.34	0.29	0.39	0.62	0.19	0.35	Classical ML	0.7857	0.7857	0.7857	0.7857
DL: BiLSTM	0.34	0.29	0.39	0.62	0.19	0.35	Deep Learning	0.3447	0.3447	0.3447	0.3447
DL: CNN	0.34	0.29	0.39	0.62	0.19	0.35	Deep Learning	0.2151	0.2151	0.2151	0.2151
weighted avg	0.34	0.29	0.39	0.62	0.19	0.35		0.27	0.27	0.27	0.27

```

# — Comprehensive Comparison Visualization —————
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('ML vs Deep Learning – Comprehensive Comparison', fontsize=14, for

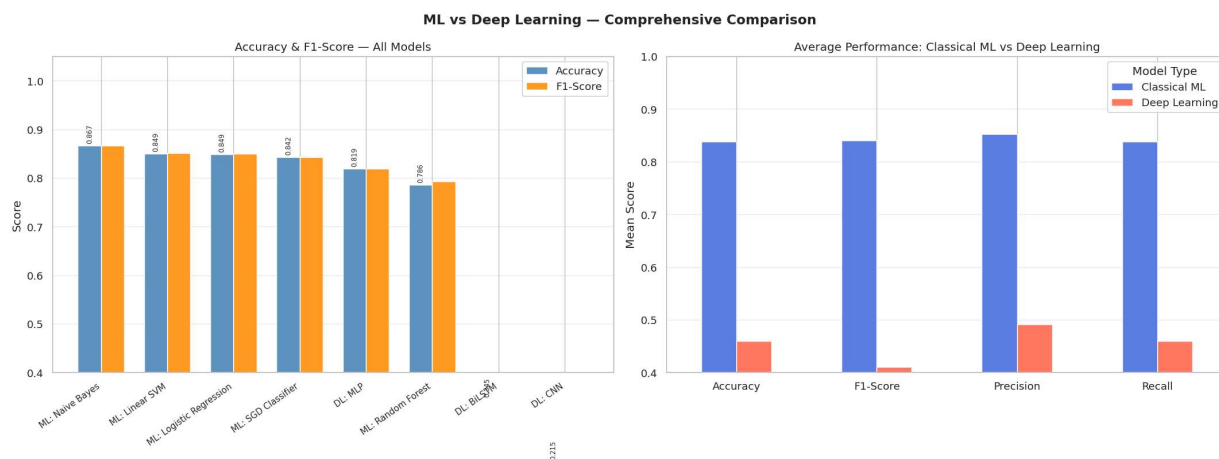
model_names = all_df.index.tolist()
acc_vals    = all_df['Accuracy'].values.astype(float)
f1_vals     = all_df['F1-Score'].values.astype(float)
bar_x       = np.arange(len(model_names))
w           = 0.35

bar1 = axes[0].bar(bar_x - w/2, acc_vals, w, label='Accuracy', color='steelblu
bar2 = axes[0].bar(bar_x + w/2, f1_vals, w, label='F1-Score', color='darkorar
axes[0].set_xticks(bar_x)
axes[0].set_xticklabels(model_names, rotation=35, ha='right', fontsize=9)
axes[0].set_ylim(0.4, 1.05)
axes[0].set_ylabel('Score')
axes[0].set_title('Accuracy & F1-Score – All Models')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3)
for bar in bar1:
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                  f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=

# Average performance: Classical ML vs Deep Learning
type_summary = all_df.groupby('Type')[['Accuracy', 'F1-Score', 'Precision', 'Recal
type_summary.T.plot(kind='bar', ax=axes[1], color=['royalblue', 'tomato'], alpha
axes[1].set_title('Average Performance: Classical ML vs Deep Learning')
axes[1].set_ylabel('Mean Score')
axes[1].set_ylim(0.4, 1.0)
axes[1].tick_params(axis='x', rotation=0)
axes[1].legend(title='Model Type')
axes[1].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```

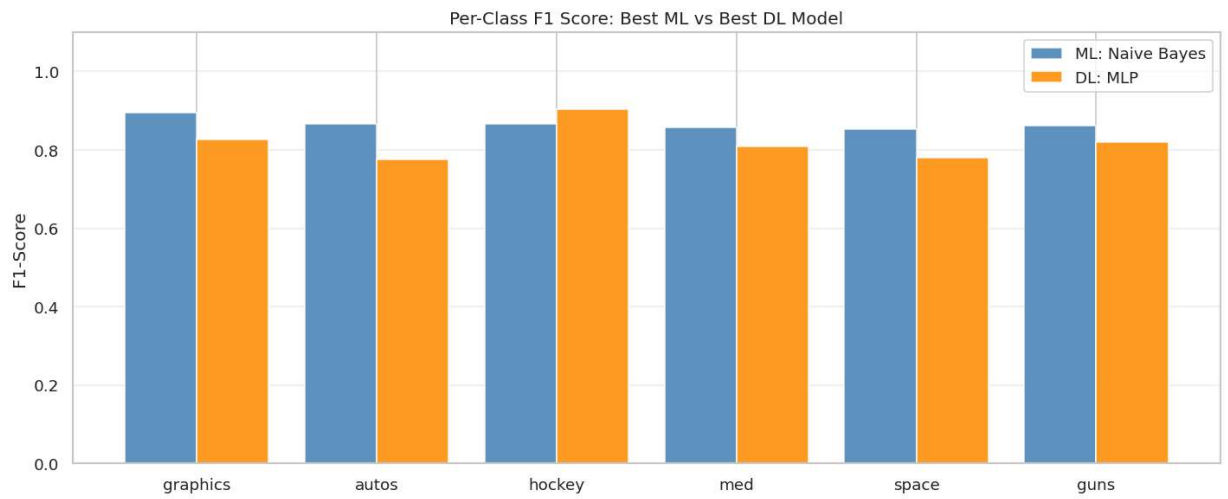


```
# — Per-Class F1 Score: Best ML vs Best DL —————
best_ml_key = all_df[all_df['Type'] == 'Classical ML'].index[0]
best_dl_key = all_df[all_df['Type'] == 'Deep Learning'].index[0]

best_ml_pred_cmp = ml_results[best_ml_key.replace('ML: ', '')]['y_pred']
best_dl_pred_cmp = dl_results[best_dl_key.replace('DL: ', '')]['y_pred']

ml_f1_per = f1_score(y_test, best_ml_pred_cmp, average=None)
dl_f1_per = f1_score(y_test, best_dl_pred_cmp, average=None)

x = np.arange(len(short_labels))
fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(x - 0.2, ml_f1_per, 0.4, label=best_ml_key, color='steelblue', alpha=0.4)
ax.bar(x + 0.2, dl_f1_per, 0.4, label=best_dl_key, color='darkorange', alpha=0.4)
ax.set_xticks(x)
ax.set_xticklabels(short_labels)
ax.set_ylabel('F1-Score')
ax.set_title('Per-Class F1 Score: Best ML vs Best DL Model')
ax.set_ylim(0, 1.1)
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Section 8: Error Analysis & Model Interpretability

```
# — Misclassified Examples Analysis —————
best_ml_pred_final = ml_results[best_ml_name]['y_pred']
misclassified_idx = np.where(best_ml_pred_final != y_test)[0]

print(f"Total misclassified: {len(misclassified_idx)} / {len(y_test)}"
      f" ({len(misclassified_idx)/len(y_test)*100:.1f}%)")
print("\nSample misclassified examples:")
print("-" * 70)

for i in misclassified_idx[:5]:
    # Use target_names for correct label→name mapping
    true_cat = target_names[y_test[i]].split('.')[0]
    pred_cat = target_names[best_ml_pred_final[i]].split('.')[0]
    text_preview = df_test['text'].iloc[i][:150].replace('\n', ' ')
    print(f"[Idx {i}]")
    print(f" True : {true_cat}")
    print(f" Pred : {pred_cat}")
    print(f" Text : {text_preview}...")
    print()
```

Total misclassified: 312 / 2338 (13.3%)

Sample misclassified examples:

```
-----
[Idx 1]
 True : space
 Pred : guns
 Text : This is, shall we say, an overly-broad statement. In particular, are

[Idx 9]
 True : guns
 Pred : space
 Text : As far as I know, there is no FAQ for tpg. Somebody was wo

[Idx 25]
 True : autos
 Pred : med
 Text : In my opinion, the limited tort option is the best thing Casey has ever

[Idx 31]
 True : med
 Pred : hockey
 Text : ...

[Idx 50]
 True : med
 Pred : guns
 Text : Do you swim nude in a public swimming pool? :) I doubt sperm can
```

```
# — Top Discriminative Features per Class —————
# Logistic Regression coefficients reveal which words most drive each predictic
# coef_[i] corresponds to target_names[i] (alphabetical label order).
```

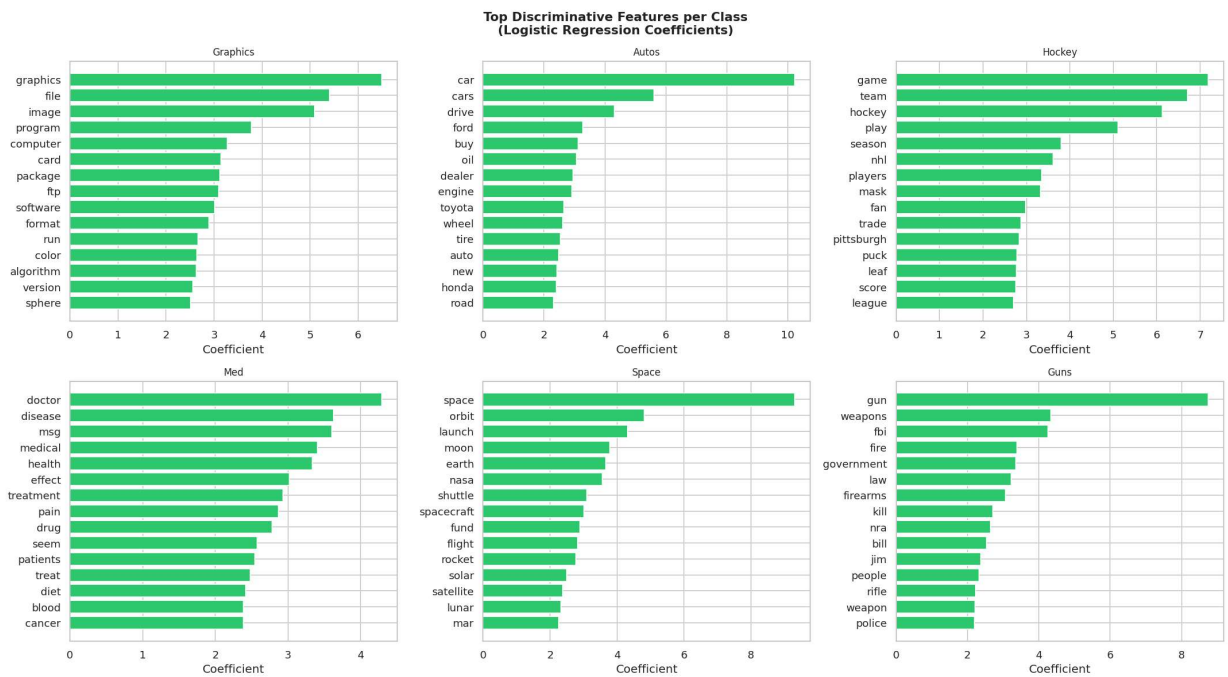
```
lr_model = ml_models['Logistic Regression']
feat_names_tfidf = tfidf_vectorizer.get_feature_names_out()

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Top Discriminative Features per Class\n(Logistic Regression Coeff
             fontsize=13, fontweight='bold')

# Iterate over target_names – ensures coef_[idx] aligns with correct category
for idx, (ax, cat) in enumerate(zip(axes.flatten(), target_names)):
    coefs = lr_model.coef_[idx] # idx matches alphabetical label
    top_pos_idx = np.argsort(coefs)[-15:][::-1]
    words = [feat_names_tfidf[i] for i in top_pos_idx]
    scores = [coefs[i] for i in top_pos_idx]
    colors_feat = ['#2ecc71' if s > 0 else '#e74c3c' for s in scores]
    ax.barh(words[::-1], scores[::-1], color=colors_feat[::-1])
    ax.set_title(cat.split('.')[0].capitalize(), fontsize=10)
    ax.set_xlabel('Coefficient')
    ax.axvline(0, color='black', linewidth=0.5)

plt.tight_layout()
plt.show()

# Sanity check: top word per class should match expected domain vocabulary
print("Sanity check – top discriminative word per class:")
for idx, cat in enumerate(target_names):
    top_word = feat_names_tfidf[np.argmax(lr_model.coef_[idx])]
    print(f" {cat.split('.')[0]:15s} → '{top_word}'")
```

Sanity check – top discriminative word per class:

graphics → 'graphics'
 autos → 'car'
 hockey → 'game'
 med → 'doctor'
 space → 'space'
 guns → 'gun'

Section 9: Live Prediction Demo

```
# — Predict on Custom Text —————
def predict_category(text, use_dl=False, dl_model_name='MLP'):
    """
```

Predict the newsgroup category of a given text string.

Key fix: Uses target_names[pred] for correct label→name mapping.
 sklearn assigns labels alphabetically, NOT in the CATEGORIES list order.
 So pred=0 → comp.graphics, pred=2 → rec.sport.hockey, etc.
 """

```
processed = full_preprocess(text)

if not use_dl:
    vec = tfidf_vectorizer.transform([processed])
    pred = best_lr.predict(vec)[0]          # integer label (alphabetical order)
    prob = best_lr.predict_proba(vec)[0]    # probabilities in same order
else:
    seq = keras_tokenizer.texts_to_sequences([processed])
    pad = pad_sequences(seq, maxlen=MAX_LEN, padding='post')
    prob = dl_models[dl_model_name].predict(pad, verbose=0)[0]
    pred = np.argmax(prob)

print(f"Input      : {text[:100]}")
print(f"Predicted: {target_names[pred]}")    # target_names[pred] = CORRECT
print("\nConfidence scores:")
# zip with target_names – same alphabetical order as label indices
for cat, p in sorted(zip(target_names, prob), key=lambda x: -x[1]):
    bar = '█' * int(p * 30)
    print(f" {cat.split('.')[0]:15s} {bar:<30} {p:.4f}")
print()
```

— Test sentences (each clearly belongs to one of the 6 categories) —

```
test_sentences = [
    "The hockey team scored three goals in the third period, winning the Stanley Cup."
    "NASA launched a new satellite to explore deep space radiation and magnetic fields."
    "The doctor prescribed antibiotics for the bacterial infection in the patient."
    "The new GPU renders 3D graphics with unprecedented ray tracing performance."
    "The senator argued that gun control laws violate the Second Amendment rights."
    "The car engine requires a turbocharger replacement after 100,000 miles."
]
```

```
print("=" * 70)
print("LIVE PREDICTION DEMO – Logistic Regression (TF-IDF)")
print("=" * 70)
for s in test_sentences:
    predict_category(s, use_dl=False)
    print("-" * 70)
```